

序列的通用操作+元组+列表

邓擎琮
北京师范大学

序列（sequence）-有顺序的容器

- 序列：数据元素按次序排列，可以通过索引下标访问的对象。

元组

- 元素可为不同类型，不可变类型，()为界

列表

- 元素可为不同类型，可变类型，[]为界

字符串

- 元素均为字符，不可变类型，单引号/双引号/三引号为界

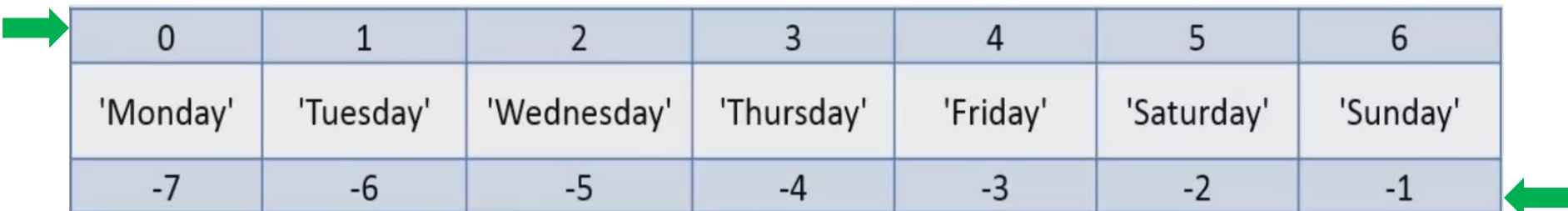
字节序列

- 元素为二进制编码，可变/不可变类型，b"/b" "/b"" ""/b"""" """"为界

序列的基本操作

■ 索引访问操作

```
week=('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday')
```



0	1	2	3	4	5	6
'Monday'	'Tuesday'	'Wednesday'	'Thursday'	'Friday'	'Saturday'	'Sunday'
-7	-6	-5	-4	-3	-2	-1

索引越界 `IndexError: tuple index out of range`

序列的基本操作2

■ for 循环遍历组成成员

```
>>> for c in 'hello':  
    print(c)
```

```
h  
e  
l  
l  
o
```

```
>>> for i in (1,2,3):  
    print(i)
```

```
1  
2  
3
```

```
>>> for t in [(1,2), (3,4), (5,6)]:  
    print(t,t[0],t[1])
```

```
(1, 2) 1 2  
(3, 4) 3 4  
(5, 6) 5 6
```

序列的基本操作3

- 切片操作
- 连接操作
- 重复操作
- 成员关系操作
- 比较运算操作
- 序列可用的内置函数
- 序列拆封操作

切片操作

- 通过切片(slice)操作，可截取序列的一部分元素。

- 基本形式：

起 终 步长
↓ ↓ ↓
s[i:j] 或者 s[i:j:k] **[i,j)**

切片操作示例

```
week=('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday')
```

0	1	2	3	4	5	6
'Monday'	'Tuesday'	'Wednesday'	'Thursday'	'Friday'	'Saturday'	'Sunday'
-7	-6	-5	-4	-3	-2	-1

```
>>> print(week[:5], '\n', week[-2:], '\n', week[:5:2], '\n', week[::-1])
('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday')
('Saturday', 'Sunday')
('Monday', 'Wednesday', 'Friday')
('Sunday', 'Saturday', 'Friday', 'Thursday', 'Wednesday', 'Tuesday', 'Monday')

>>> print(week[5:100], '\n', week[7:100])
('Saturday', 'Sunday')
()
```

索引越界不报错

连接操作

■ 连接+：s1+s2

```
>>> t1=(1,2)
>>> t2=('a','b')
>>> t1+t2
(1, 2, 'a', 'b')
```

```
>>> t3=['a','b']
>>> t1+t3
```

Traceback (most recent call last):

```
File "<pyshell#22>", line 1, in <module>
    t1+t3
```

TypeError: can only concatenate tuple (not "list") to tuple

```
>>> t3+t1
```

Traceback (most recent call last):

```
File "<pyshell#23>", line 1, in <module>
    t3+t1
```

TypeError: can only concatenate list (not "tuple") to list

重复操作

- 重复*: $s*n$ 或者 $n*s$

```
>>> t1*2
```

```
(1, 2, 1, 2)
```

```
>>> 'Apple '*3
```

```
'Apple Apple Apple '
```

成员关系操作

- **in**, **not in**: 判断一个元素是否在序列中

```
>>> lst=[1,2,3,2,1]
```

```
>>> 1 in lst
```

```
True
```

```
>>> 4 not in lst
```

```
True
```

```
>>> s='good better best'
```

```
>>> 'G' in s
```

```
False
```

```
>>> 'be' in s
```

```
True
```

成员关系操作2

- **s.count(value)** : value在序列中出现的次数

```
lst=[1,2,3,2,1]      s='good better best'
```

```
>>> lst.count(1)
```

```
2
```

```
>>> lst.count(0)
```

```
0
```

```
>>> s.count('o')
```

```
2
```

```
>>> s.count('be')
```

```
2
```

```
>>> s.count('be', 4, 11)
```

```
1
```

成员关系操作3

- `s.index(value, [start, [stop]])` : `value`在序列指定范围 `[start,stop)`中第一次出现的下标

```
lst=[1,2,3,2,1]    s='good better best'
```

```
>>> lst.index(3)
```

```
2
```

```
>>> lst.index(1,1)
```

```
4
```

```
>>> s.index(' ')
```

```
4
```

```
>>> s.index('be',6)
```

```
12
```

成员关系操作3

- `s.index(value, [start, [stop]])` : `value`在序列指定范围`[start,stop)`中没有会报错!

```
lst=[1,2,3,2,1]    s='good better best'
```

```
>>> lst.index(4)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#71>", line 1, in <module>
```

```
    lst.index(4)
```

```
ValueError: 4 is not in list
```

```
>>> s.index('be',6,13)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#70>", line 1, in <module>
```

```
    s.index('be',6,13)
```

```
ValueError: substring not found
```

比较运算操作

- `>`, `<`, `>=`, `<=`, `!=`, `==`, `is`, `is not`

```
>>> (1,2,3)>[1,2]
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#47>", line 1, in <module>
```

```
    (1,2,3)>[1,2]
```

```
TypeError: unorderable types: tuple() > list()
```

```
>>> (1,2,3)!='abc'
```

```
True
```

```
>>> (1,2,3)=='abc'
```

```
False
```

```
>>> (1,2,3)>('a','b')
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#51>", line 1, in <module>
```

```
    (1,2,3)>('a','b')
```

```
TypeError: unorderable types: int() > str()
```

比较运算操作

```
>>> 'apple' < 'banana'
```

```
True
```

```
>>> b'abc' > b'ABC'
```

```
True
```

```
>>> [1, 3, [2, 4]] > [1, 3, [3, 3]]
```

```
False
```

```
>>> t1=t2=[1, 2]
```

```
>>> t1 is t2
```

```
True
```

```
>>> ('apple' < 'banana') and ((1, 2) < (1, 3))
```

```
True
```

序列可用内置函数

len	sorted
reversed	min
max	sum
enumerate	zip
all	any

类型转换函数：

str	list
tuple	bytes
bytearray	

后续还有：
map、filter、
reduce

序列可用内置函数-len

■ 求长度

```
>>> a=[1,3,5,[7,9]]
>>> b=(1,2,3,[],(),)
>>> c='abcd'
>>> d=b'abc'
>>> print(len(a),len(b),len(c),len(d))
4 5 4 3
```

序列可用内置函数-**sorted**

- 要求元素类型相同，不改动原序列，**返回**一个排序后的**列表**

sorted(iterable, key=None, reverse=False)

```
>>> s1=(4,1,2,3)
```

```
>>> sorted(s1)
```

```
[1, 2, 3, 4]
```

```
>>> s1
```

```
(4, 1, 2, 3)
```

```
>>> sorted(s1, reverse=True)
```

```
[4, 3, 2, 1]
```

序列可用内置函数-sorted

- 要求元素类型相同，不改动原序列，**返回**一个排序后的**列表**

sorted(iterable, key=None, reverse=False)

```
>>> sorted(('apple', 'banana', 'pear'))  
['apple', 'banana', 'pear']  
>>> sorted(('apple', 'banana', 'pear'), key=len)  
['pear', 'apple', 'banana']
```

序列可用内置函数-reversed

- 逆序，不改动原序列，返回反向迭代器（一种惰性的会消耗的可迭代对象）

```
>>> b=[1, 'a', 12, (4, 5)]
```

```
>>> reversed(b)
```

```
<list_reverseiterator object at 0x0000000002D474E0>
```

```
>>> for i in reversed(b):  
        print(i)
```

```
(4, 5)
```

```
12
```

```
a
```

```
1
```

```
>>> b
```

```
[1, 'a', 12, (4, 5)]
```

序列可用内置函数-reversed

- 如何理解“会消耗”？

```
>>> b=[1,'a',12,(4,5)]
```

```
>>> c=reversed(b)
```

```
>>> c
```

```
<list_reverseiterator object at 0x0000000002E1C438>
```

```
>>> for i in c:  
    print(i)
```

```
(4, 5)
```

```
12
```

```
a
```

```
1
```

只支持一遍访问！

```
>>> for i in c:  
    print(i)
```

```
>>>
```

序列可用内置函数-min/max/sum

- `min()`: 返回序列的最小值
 - `max()`: 返回序列的最大值
 - `sum()`: 返回元组或列表各元素之和。 不能有非数值元素
- } 元素类型相同

```
>>> min([1,2,'a',4])
Traceback (most recent call last):
  File "<pyshell#61>", line 1, in <module>
    min([1,2,'a',4])
TypeError: unorderable types: str() < int()
```

```
>>> l=[1,2,3,4]
>>> print('min={},max={},sum={}'.format(min(l),max(l),sum(l)))
min=1,max=4,sum=10
```

序列可用内置函数-enumerate

- `enumerate(iterable, start=0)`: 返回元素为元组(计数, 元素)的迭代器

```
>>> seasons=['Spring', 'Summer', 'Fall', 'Winter']
```

```
>>> enumerate(seasons)
```

```
<enumerate object at 0x00000000034CAF78>
```

```
>>> for i in enumerate(seasons):  
    print(i)
```

```
(0, 'Spring')
```

```
(1, 'Summer')
```

```
(2, 'Fall')
```

```
(3, 'Winter')
```

```
>>> list(enumerate(seasons, 1))
```

```
[(1, 'Spring'), (2, 'Summer'), (3, 'Fall'), (4, 'Winter')]
```

序列可用内置函数-**zip**

- `zip(iter1 [,iter2 [...]])`: 拼接多个对象`iter1`、`iter2...`的元素，返回一个迭代器，其元素为各对象元素组成的元组。

```
>>> zip((1,2,3), 'abc', ['one', 'two', 'three'])  
<zip object at 0x00000000034D29C8>
```

```
>>> for i in zip((1,2,3), 'abc', ['one', 'two', 'three']):  
    print(i)
```

```
(1, 'a', 'one')  
(2, 'b', 'two')  
(3, 'c', 'three')
```

```
>>> list(zip((1,2,3,4), 'abc'))  
[(1, 'a'), (2, 'b'), (3, 'c')]
```


序列可用内置函数-all/any

- 判断序列的元素是否全部和部分为True

```
>>> all((None, 0, False))
False
>>> any((None, 0, False))
False
>>> all((1, 2, 0))
False
>>> any([1, 2, 0])
True
```

序列可用内置函数-类型转换

```
>>> list('Hello')
['H', 'e', 'l', 'l', 'o']
>>> tuple('world')
('w', 'o', 'r', 'l', 'd')
>>> str([1,2,3])
'[1, 2, 3]'
>>> str(reversed('abcd'))
'<reversed object at 0x00000000034D8BA8>'
```

序列拆封操作

- 使用赋值语句，可以将序列拆封，赋值给多个变量：

变量1,变量2,...,变量n = 序列

```
>>> data=(1001, '张三', (80,79,92))
>>> sn,name,scores=data
>>> print(sn,'\n',name,'\n',scores)
1001
张三
(80, 79, 92)
```

```
>>> sn,name,(chinese,math,english)=data
>>> math
79
```

序列拆封操作

- 使用赋值语句，可以将序列拆封，赋值给多个变量：

变量1,变量2,...,变量n = 序列

```
>>> data=(1001, '张三', (80,79,92))
>>> sn,name,scores=data
>>> print(sn,'\n',name,'\n',scores)
1001
张三
(80, 79, 92)
```

```
>>> sn,name,(chinese,math,english)=data
>>> math
79
```

```
>>> sn,name,chinese,math,english=data
```

```
Traceback (most recent call last):
```

```
File "<pyshell#125>", line 1, in <module>
    sn,name,chinese,math,english=data
```

```
ValueError: need more than 3 values to unpack
```

序列拆封操作2

- 通过***变量**，将多个元素打包赋值给该变量，***变量只能出现一次**

```
>>> first,*middle,last=range(10)
>>> print(first,'\n',middle,'\n',last)
0
[1, 2, 3, 4, 5, 6, 7, 8]
9
```

```
>>> first,second,third,*last=range(10)
>>> last
[3, 4, 5, 6, 7, 8, 9]
```

```
>>> *first,middle,last=range(10)
>>> first
[0, 1, 2, 3, 4, 5, 6, 7]
```

序列拆封操作3

- 可使用临时变量

```
>>> _, b, _ = (1, 2, 3)
>>> b
2
```

```
>>> record=('张三', 'sz@abc.com', '58801234', '13912345678')
>>> name, _, *phones=record
>>> phones
['58801234', '13912345678']
```

序列 (sequence)

- 序列：数据元素按次序排列，可以通过索引下标访问的对象。

元组

- 元素可为不同类型，不可变类型，() 为界

列表

- 元素可为不同类型，可变类型，[] 为界

字符串

- 元素均为字符，不可变类型，单引号/双引号/三引号为界

字节序列

- 元素为二进制编码，可变/不可变类型，b"/b" "/b"" ""/b"""" """"为界

元组

■ 元组的定义:

- (x1,[x2,x3,...,xn])或者 x1,[x2,x3,...,xn]

- () 或 tuple(): 创建一个空元组

- tuple(iterable)

```
>>> t1=1,2,3
```

```
>>> print(type(t1), '\t', t1)
```

```
<class 'tuple'>          (1, 2, 3)
```

```
>>> 1,          >>> (1)
(1,)           1
```

```
>>> print(tuple(), type(tuple()), (), type(()))
() <class 'tuple'> () <class 'tuple'>
```

```
>>> print(tuple(range(5)), tuple('abc'))
(0, 1, 2, 3, 4) ('a', 'b', 'c')
```


元组的基本操作

■ 元组的基本操作：序列的通用操作

```
>>> t1=(1,2,1)
>>> t2=('x','y','z')
>>> len(t1)
3
>>> len(t2)
3
>>> t1[0:2]
(1, 2)
>>> max(t1)
2
>>> min(t2)
'x'
>>> sum(t1)
4
>>> t1+t2
(1, 2, 1, 'x', 'y', 'z')
```

列表

■ 列表的定义

- `[ x1,[x2,x3,...,xn] ]`
- `[]` 或 `list()`: 创建一个空列表
- `list(iterable)`

```
>>> lst=[1,2,3]
```

```
>>> print(type(lst), lst)
```

```
<class 'list'> [1, 2, 3]
```

```
>>> print(list(), type(list()), [], type([]))
```

```
[] <class 'list'> [] <class 'list'>
```

```
>>> print(list(range(5)), list('abc'))
```

```
[0, 1, 2, 3, 4] ['a', 'b', 'c']
```

列表的基本操作

- 序列的通用操作
- 通过赋值语句修改元素 和 **del**删除元素

```
>>> s=[1,2,3,4,5,6,7,8]
>>> s[1]='a'; s
[1, 'a', 3, 4, 5, 6, 7, 8]
>>> s[2]=[]; s
[1, 'a', [], 4, 5, 6, 7, 8]
```

列表的基本操作

- 序列的通用操作
- 通过赋值语句修改元素 和 **del**删除元素

```
>>> s=[1,2,3,4,5,6,7,8]
>>> s[1]='a'; s
[1, 'a', 3, 4, 5, 6, 7, 8]
>>> s[2]=[]; s
[1, 'a', [], 4, 5, 6, 7, 8]
```

```
>>> del s[3]; s
[1, 'a', [], 5, 6, 7, 8]
```

```
>>> del s[1:3]; s
[1, 5, 6, 7, 8]
```

```
>>> s[:2]='b'; s
['b', 6, 7, 8]
```

```
>>> s[1:3]=[]; s
['b', 8]
```

相当于 **del** s[1:3]

列表的基本操作2

■ list对象的方法

`s=[1,2,3]`

方法	说明	示例
<code>s.append(x)</code>	把对象x追加到s的尾部	<code>s.append('a')</code> #s=[1,2,3,'a'] <code>s.append([1,2])</code> #s=[1,2,3,[1,2]]
<code>s.clear()</code>	删除所有元素	<code>s.clear()</code> #s=[]
<code>s.copy()</code>	拷贝	<code>s1=s.copy()</code> #s1=[1,2,3] # id(s1) != id(s)
<code>s.extend(t)</code>	把t附加到s的尾部, 相当于s+=t	<code>s.extend(4)</code> #s=[1,2,3,4] <code>s.extend('ab')</code> #s=[1,2,3,'a','b']
<code>s.insert(i,x)</code>	在下标i位置插入x	<code>s.insert(1,4)</code> #s=[1,4,2,3] <code>s.insert(8,4)</code> #s=[1,2,3,4] <code>s.insert(-10,4)</code> #s=[4,1,2,3]

列表的基本操作3

`s=[1,2,3]`

方法	说明	示例
<code>s.pop([i])</code>	返回并移除下标 <i>i</i> 位置的元素， <i>i</i> 省略时为最后元素	<code>s.pop()</code> #输出3， <code>s=[1,2]</code> <code>s.pop(0)</code> #输出1， <code>s=[2,3]</code> <code>s.pop(4)</code> # <code>IndexError: pop index out of range</code>
<code>s.remove(x)</code>	移除列表中第一次出现的 <i>x</i>	<code>s.remove(1)</code> # <code>s=[2,3]</code> <code>s.remove('a')</code> # <code>ValueError: list.remove(x): x not in list</code>
<code>s.count(x)</code>	<i>x</i> 在列表中出现的次数	<code>s.count(2)</code> #输出1
<code>s.index(x, [start, [stop]])</code>	<i>x</i> 在列表[start,stop)范围中第一次出现的下标	<code>s.index(1)</code> #输出0 <code>s.index(1,1)</code> # <code>ValueError: 1 is not in list</code>
<code>s.reverse()</code>	列表反转	<code>s.reverse()</code> # <code>s=[3,2,1]</code>
<code>s.sort()</code>	列表排序	<code>s.sort(reverse=True)</code> # <code>s=[3,2,1]</code>

列表解析表达式

- 通过列表解析可以简单高效处理可迭代对象，并生成结果列表。
- 格式：

[express for i_1 in 可迭代对象1...for i_N in 可迭代对象N [if condition]]

```
>>> [i for i in range(10)]  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> a=[]  
>>> for i in range(10):  
    a.append(i)
```

列表解析表达式2

```
>>> [i**2 for i in range(10) if i%2==0]  
[0, 4, 16, 36, 64]
```

```
>>> a=[]
```

```
>>> for i in range(10):  
    if i%2==0:  
        a.append(i**2)
```

```
>>> [(x+y,x*y) for x in range(2) for y in range(2,4)]  
[(2, 0), (3, 0), (3, 2), (4, 3)]
```

```
>>> a=[]
```

```
>>> for x in range(2):  
    for y in range(2,4):  
        a.append((x+y,x*y))
```


列表解析表达式3

■ 用列表解析生成字典和集合

```
>>> a=[('小黑','领导',30000),\
        ('小白','职员',10000),\
        ('小蓝','职员',5000)]
>>> {i[0]:i[2] for i in a}
{'小白': 10000, '小蓝': 5000, '小黑': 30000}

>>> {i[0] for i in a if i[2]>=10000}
{'小白', '小黑'}
```

```
>>> (i for i in range(10))    生成器表达式
<generator object <genexpr> at 0x00000000034CAC60>
```